Lekcja 9: Praca z tekstem, serializacja i systemem plików

`#lekcja` `#python` `#pliki` `#json` `#csv` `#kodowanie`

W tej lekcji nauczymy się, jak sprawić, by nasze programy komunikowały się ze światem zewnętrznym. Opanujemy kluczowe umiejętności każdego programisty: pracę z plikami, zrozumienie kodowania znaków oraz przetwarzanie danych w popularnych formatach, takich jak JSON, CSV i Excel. To fundament, który pozwala tworzyć aplikacje przechowujące dane i wchodzące w interakcje z innymi systemami.

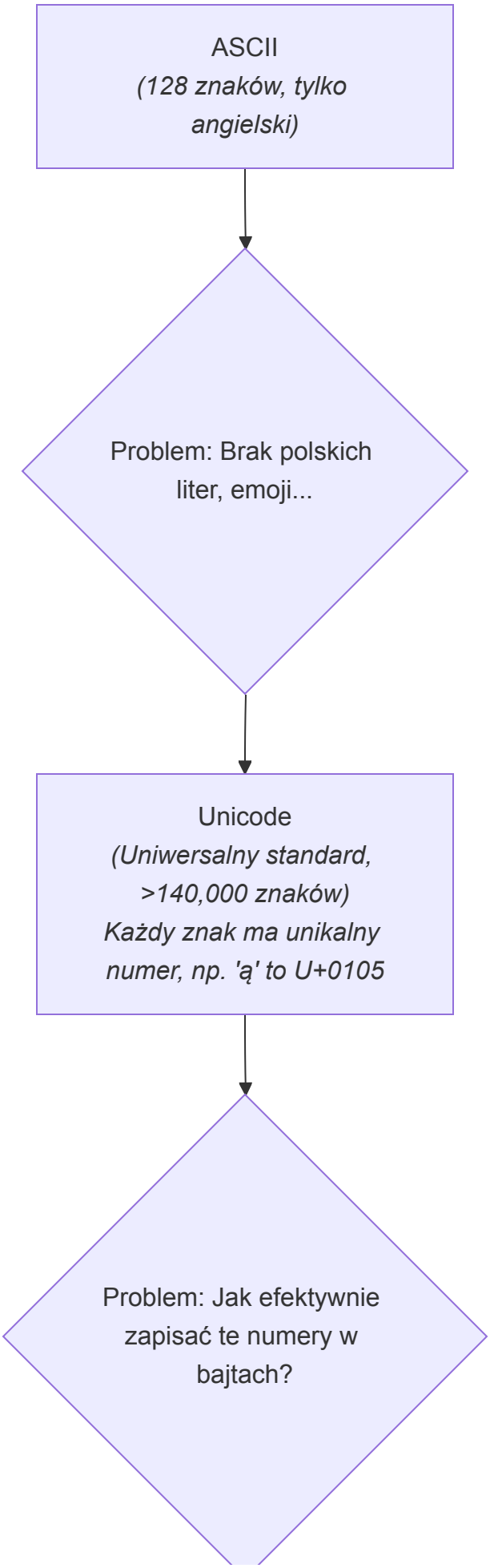
1. Kodowanie znaków

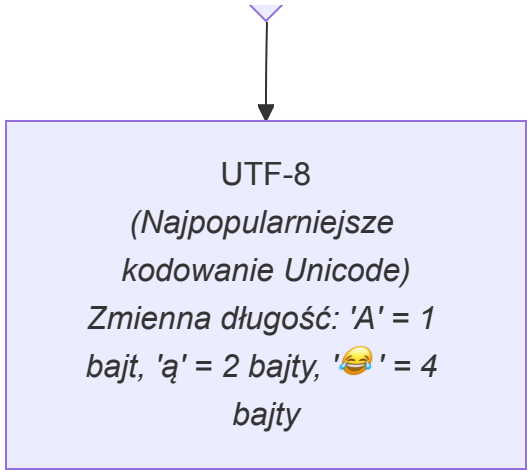
Czym jest kodowanie?

> **Definition**
>
> Kodowanie znaków to system, który przypisuje każdemu znakowi (literze, cyfrze, symbolowi) unikalną liczbę, którą komputer może zapisać w postaci bajtów. Jest to "słownik" tłumaczący świat ludzi (tekst) na świat maszyn (liczby binarne).

Dlaczego to ważne? Bez wspólnego standardu kodowania, ten sam ciąg bajtów mógłby być interpretowany jako "cześć" w jednym systemie, a jako "Äĺđŝ" w innym.

Ewolucja: ASCII → Unicode → UTF-8


ASCII
(128 znaków, tylko angielski)
Problem: Brak polskich liter, emoji...
Unicode
(Uniwersalny standard, >140,000 znaków)
Każdy znak ma unikalny numer, np. 'ą' to U+0105
Problem: Jak efektywnie zapisać te numery w bajtach?



- **ASCII:** Stary standard, który zawierał tylko litery angielskie, cyfry i podstawowe symbole.
- **Unicode:** Ogromna tabela, która przypisuje unikalny numer (tzw. *code point*) każdemu znakowi z niemal wszystkich języków świata.
- **UTF-8:** Najpopularniejszy sposób **zapisywania** numerów Unicode w bajtach. Jest wstecznie kompatybilny z ASCII i bardzo wydajny.

> **Tip**
>
> Złota zasada: Zawsze, gdy pracujesz z plikami tekstowymi, jawnie określaj kodowanie, najczęściej encoding="utf-8". Unikniesz w ten sposób problemów z polskimi znakami.

2. Praca z plikami

Funkcja `open()` i menedżer kontekstu `with`

Najlepszym i najbezpieczniejszym sposobem pracy z plikami w Pythonie jest użycie **menedżera kontekstu `with`**. Gwarantuje on, że plik zostanie automatycznie zamknięty, nawet jeśli w trakcie operacji wystąpi błąd.

```
# Składnia: with open(ścieżka, tryb, encoding) as nazwa_zmiennej:
with open("moj_plik.txt", "w", encoding="utf-8") as f:
    f.write("Witaj, świecie!")
# Plik jest automatycznie zamykany tutaj
```

Najważniejsze tryby otwarcia (`mode`):

Tryb	Opis	Działanie, jeśli plik nie istnieje	Działanie, jeśli plik istnieje

`'r'`	**Odczyt** (Read) - domyślny	Błąd `FileNotFoundError`	Ustawia wskaźnik na początku
`'w'`	**Zapis** (Write)	Tworzy nowy plik	**Kasuje zawartość** i pisze od nowa
`'a'`	**Dopisywanie** (Append)	Tworzy nowy plik	Ustawia wskaźnik na końcu
`'x'`	**Wyłączne tworzenie**	Tworzy nowy plik	Błąd `FileExistsError`
`'+'`	Dodaje możliwość **odczytu i zapisu** (np. `'r+'`, `'w+'`)	-	-
`'b'`	Tryb **binarny** (dla plików nietekstowych, np. obrazów)	-	-

Odczyt z pliku

```
with open("dane.txt", "r", encoding="utf-8") as f:
    # 1. Odczyt całego pliku na raz (dla małych plików)
    caly_tekst = f.read()

    # 2. Odczyt linia po linii (najlepsze dla dużych plików)
    f.seek(0) # Wracamy na początek pliku
    for linia in f:
        print(linia.strip()) # .strip() usuwa białe znaki, w tym '\n'

    # 3. Odczyt wszystkich linii do listy
    f.seek(0)
    wszystkie_linie = f.readlines()
```

Zapis do pliku

```
dane_do_zapisu = ["Pierwsza linia\n", "Druga linia\n"]

with open("nowy_plik.txt", "w", encoding="utf-8") as f:
    f.write("To jest pojedyncza linia.\n")
    f.writelines(dane_do_zapisu) # Zapisuje listę stringów
```

Praca ze ścieżkami - moduł `pathlib`

Zamiast ręcznie składać ścieżki jako stringi, warto używać modułu `pathlib`, który jest bardziej elegancki i niezależny od systemu operacyjnego.

```python
from pathlib import Path

# Tworzenie ścieżki
sciezka_do_pliku = Path("dane") / "raporty" / "raport.txt"

# Tworzenie folderów, jeśli nie istnieją
sciezka_do_pliku.parent.mkdir(parents=True, exist_ok=True)

# Zapis do pliku
with open(sciezka_do_pliku, "w") as f:
    f.write("To jest raport.")

print(f"Plik istnieje: {sciezka_do_pliku.exists()}")
```

3. Praca z danymi zewnętrznymi (Serializacja)

> **Serializacja** to proces konwersji obiektu Pythona (np. słownika, listy) na format, który można zapisać do pliku lub przesłać przez sieć (np. tekst). **Deserializacja** to proces odwrotny.

JSON (JavaScript Object Notation)

Lekki, tekstowy format danych, czytelny dla człowieka. Jego struktura jest niemal identyczna jak słowników i list w Pythonie.

```python
import json

# Dane w Pythonie (słownik)
uzytkownik = {
    "imie": "Zofia",
    "wiek": 28,
    "miasta": ["Gdańsk", "Warszawa"],
    "aktywny": True
}

# --- Serializacja (Python -> JSON) ---
# Zapis do pliku
with open("uzytkownik.json", "w", encoding="utf-8") as f:
    # indent - tworzy ładne wcięcia
    # ensure_ascii=False - kluczowe dla polskich znaków!
    json.dump(uzytkownik, f, indent=4, ensure_ascii=False)

# --- Deserializacja (JSON -> Python) ---
# Odczyt z pliku
```

```
with open("uzytkownik.json", "r", encoding="utf-8") as f:
    dane_z_pliku = json.load(f)

print(dane_z_pliku["imie"]) # Zofia
```

CSV (Comma-Separated Values)

Prosty format tekstowy, idealny do danych tabelarycznych. Każdy wiersz to rekord, a wartości są oddzielone separatorem (zwykle przecinkiem).

```
import csv

dane_csv = [
    ["Imię", "Wiek", "Miasto"],
    ["Anna", 25, "Gdańsk"],
    ["Piotr", 32, "Kraków"]
]

# --- Zapis do pliku CSV ---
with open("dane.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, delimiter=';') # Możemy zmienić separator
    writer.writerows(dane_csv)

# --- Odczyt z pliku CSV ---
with open("dane.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter=';')
    for wiersz in reader:
        print(wiersz)
```

Excel (`.xlsx`)

Do pracy z plikami Excela potrzebna jest zewnętrzna biblioteka, np. openpyxl.

Należy ją najpierw zainstalować: pip install openpyxl

```
from openpyxl import Workbook, load_workbook

# --- Zapis do pliku Excel ---
wb = Workbook()
ws = wb.active # Aktywny arkusz
ws.title = "Użytkownicy"

ws.append(["Imię", "Wiek"]) # Dodaj nagłówek
ws.append(["Anna", 25])
ws.append(["Piotr", 32])
```

```
wb.save("uzytkownicy.xlsx")

# --- Odczyt z pliku Excel ---
wb_odczyt = load_workbook("uzytkownicy.xlsx")
ws_odczyt = wb_odczyt.active
for wiersz in ws_odczyt.iter_rows(values_only=True):
    print(wiersz)
```

🧪 4. Zadania do samodzielnej pracy

1. **Dziennik użytkownika:** Napisz program, który w pętli prosi użytkownika o wpisanie jednej linii tekstu. Każda wpisana linia powinna być dopisywana (tryb `'a'`) do pliku `dziennik.txt`. Program kończy działanie, gdy użytkownik wpisze "koniec".
2. **Licznik słów:** Stwórz program, który pyta o nazwę pliku, odczytuje go, a następnie zlicza i wyświetla całkowitą liczbę słów w tym pliku. Obsłuż błąd `FileNotFoundError`, jeśli plik nie istnieje.
3. **Konfiguracja w JSON:** Stwórz słownik Pythona z ustawieniami aplikacji, np. `konfiguracja = {"uzytkownik": "admin", "motyw": "ciemny", "rozdzielczosc": [1920, 1080]}`. Zapisz ten słownik do pliku `config.json` z wcięciami i poprawnym kodowaniem polskich znaków.
4. **Odczyt konfiguracji:** Napisz program, który odczytuje plik `config.json` z poprzedniego zadania i wyświetla komunikat: `Witaj, [uzytkownik]! Twój motyw to [motyw].`
5. **Eksport do CSV:** Masz listę słowników: `produkty = [{"nazwa": "Mleko", "cena": 3.50}, {"nazwa": "Chleb", "cena": 4.20}]`. Zapisz te dane do pliku `produkty.csv`, gdzie pierwszy wiersz to nagłówki ("nazwa", "cena").
6. **Import z CSV:** Napisz program, który odczytuje plik `produkty.csv` i oblicza sumę cen wszystkich produktów. Użyj `csv.DictReader`, aby łatwiej odwoływać się do kolumn po nazwach.
7. **Tworzenie struktury folderów:** Użyj modułu `pathlib`, aby napisać skrypt, który tworzy strukturę folderów: `Projekt/src`, `Projekt/data`, `Projekt/docs`.
8. **Wyszukiwarka logów:** Wyobraź sobie, że masz duży plik `log.txt`. Napisz program, który pyta użytkownika o szukane słowo (np. "ERROR") i zapisuje wszystkie linie zawierające to słowo do nowego pliku `wyniki_wyszukiwania.txt`.
9. **Prosty arkusz kalkulacyjny:** Używając `openpyxl`, stwórz plik `finanse.xlsx`. W pierwszej kolumnie umieść nazwy wydatków (np. "Czynsz", "Jedzenie"), a w drugiej ich wartości. W komórce poniżej wartości oblicz i wstaw sumę wszystkich wydatków, używając formuły Excela (np. `=SUM(B1:B2)`).
10. **Mini-projekt: Lista zadań:** Stwórz prostą aplikację do zarządzania listą zadań. Program powinien:
    - Przy starcie próbować wczytać zadania z pliku `zadania.json`.

- Pozwalać użytkownikowi dodać nowe zadanie.
- Pozwalać wyświetlić wszystkie zadania.
- Przy zamknięciu (lub na polecenie) zapisywać aktualną listę zadań do pliku `zadania.json`.